

Hidden Markov Model and Future Prediction of Stock Market

Ravindra Nath¹, Aashish Bharti², Pukhraj Gond³, Jagriti Pandey⁴

Department of Computer Science, Babasaheb Bhimrao Ambedkar University, Lucknow

¹rnkatiyar@gmail.com, ²jnvpbhaashishbharti@gmail.com, ³gondpukhraj@gmail.com, ⁴pandey.8112003@gmail.com

Abstract- Stock market is a dynamic and volatile in nature where accurately forecasting prices of shares for future return is challenging. Tackling this complexity, sophisticated statistical models such as Hidden Markov Models (HMMs) have surfaced which are quite handy in predicting future stock price patterns using its historical data. HMM is ideal in case of partially observable system, extracts hidden patterns among stock price fluctuations. The research paper finds its application by analyzing historical data (like opening and closing price, fractional price difference or intraday highs and lows) for predicting stock market prices in the future. Metrics like MAPE, RMSE, and DPA are used to evaluate the performance of this HMM model, with an aim to predict not only the magnitude but also the direction of stock prices changes. The study also provides empirical evidence supporting that HMMs are effective in stock price forecasting, which helps traders and investors to make more accurate decision. The paper also confirms that HMM scaling over the stock dataset for the other predictions across multiple stocks is possible and makes a step towards providing a generic technique for future analysis of the stock market.

Keywords: Hidden Markov Models, Stock Market Prediction, Time Series Forecasting, Mean Absolute Percentage Error (MAPE), Directional Prediction Accuracy(DPA), Root Mean Squared Error (RMSE).

I. INTRODUCTION

PREDICTION of the stock market has been an area that has captivated the interest of institutional investors, hedge funds and proprietary trading firms for decades; markets are volatile and in the financial world there is always a figure out to predict a way to make a fortune or otherwise beat it. These advanced market players are looking to predict future price action and overall trend health in hopes of providing themselves similar predictive and edge while increasing profits from their investment.

Institutional investors include organizations such as pension funds and insurance companies which manage the large pool of money on behalf of their clients/beneficiaries. Some of these investors are pensions funds who needs to ensure that they pay their current staff and retiring employees now and in 10 years — while meeting other obligations like social security, payroll, etc. The stock market is predictable so they can target more optimal opportunities and reduce exposure to risks, likely resulting in better outcomes for their managed portfolio thus fulfilling their fiduciary duties [1].

On the other hand, hedge funds are simply private investment partnerships which pool funds from accredited investors. Hedge fund managers look to deliver substantial returns on average, often regardless of market trends through combining varied investment processes. True to be told, accurate prediction is the base for hedge funds to spot and exploit mispriced securities or market inefficiencies among other trading strategies that attract investors and piles in huge profit [2].

These savant market participants employ a gamut of quantitative and qualitative methodologies to come up with forecasts in the stock market.

Algorithmic trading strategies use computer algorithms which analyze a lot of market data, identify patterns and generate trade signals. Most automated trading systems are designed to trade automatically, the trader having implemented

very complex algorithms to use mathematical and statistical techniques. By now, U.S. stock trading volume from algorithmic trading represents an estimated 50% [3].

Algorithmic trading has advantages as it can provide access to data and execute trades quickly, especially for institutional traders but this doesn't take the discretion out of investors altogether. Algorithmic trading is frequently presumed to be successful as a result of deep research, good infrastructure, and significant resources that might not be easily accessible by the average individual retail investor. Additionally, markets can change quickly, which leaves algorithms unable to perform or potentially making the loss much larger. Algorithmic trading is also perhaps more risky for retail investors who do not have the pockets and risk analysis and management skills of institutions.

There emerged several tactics for algorithmic trading over the years. Hidden Markov Models (HMMs) have come up as a strong tool in the prediction of time series data, and we want to apply them here as well, so they are ideal choice for such problems where stock market is to be predicted.

II. LITERATURE REVIEW

Aditya Gupta and Bhuwan Dhingra in [4] used a method to predict the next day's Close using Hidden Markov Model. Stocks of four different companies viz. TATA Steel, Apple Inc., IBM Corporation and Dell Inc. Were taken into consideration. The input parameters were Open, Close, High and Low. The model for one stock was assumed to be independent of other stocks. The model was first trained for a period of 7 months. The model was tested using MAPE values.

Abhishek, Khairwa, Pratap and Prakash in [5] predicted the stock rate using feed forward architecture. The stocks of Microsoft Corporation of the year 2011 were used. The parameters such as Open, High, Low, Adjacent Close and Volume were considered. Next day's Close value was

predicted using back propagation with multilayer feed forward network. Mean Squared Error (MSE) was used to check for the accuracy of the proposed system. The author also presented a literature review on application of neural network to predict stock markets.

Luo, Wu, Yan in [6] presented a novel method for the study of Shanghai Stock Exchange (SSE) index time series based on SVM regression combination model (SVR-CM) linear regression with non linear regression. Linear regression was used to extract linear features while NN algorithms were used to extract non linear features. Finally, SVM regression was used to combine all output results. Mean Absolute Percentage Error (MAPE), Root Mean Square Error (RMSE), Trend Accuracy were used to indicate the effectiveness of the proposed system. SVM-CM was found to have good learning ability and forecasting capability.

Wang and Leu in [7] developed a prediction system useful in forecasting mid-term price trend in Taiwan stock exchange weighted stock index (TSEWSI). The system was based on a recurrent neural network trained by using features extracted from Auto Regressive Integrated Moving Average (ARIMA). The input is same as the standard input given to ARIMA while the output approximates conditional mean predictor. Features were operated using difference operator. The accuracy was found good for six weeks when the training data was taken for 3 years and the testing data for 3 months.

Akinwale adio T, Arogundade O.T and Adekoya Adebayo F in [8] examined the use of error back propagation and regression analysis to predict the untranslated and translated Nigeria Stock Market Price (NSMP). The author used 5-j-1 network topology to adopt the five input variables. The j variables in network determined the number of hidden neurons needed. The translated and untranslated statements were analysed and compared. The accuracy of translated NSMP was better than untranslated NSMP.

Wang, Liu and Dou in [9] presented the first work which shows a service oriented approach to facilitate the incorporation of multiple factors in stock market volatility prediction. Multi kernel learning approach was used. The system was a two tier architecture. The top tier was dedicated to prepare the datasets from multiple information while the second tier was dedicated to the market volatility analysis and prediction. The accuracy of the prediction was obtained by checking whether the direction of the predicted volatility and actual trend are the same. The output is either up or down.

Naeini, Taremi and Hashemi in [10] used two methods to predict feed forward multi layer perceptron and Elman recurrent network. Model based on MLP NN for Tehran Stock Exchange (TSE) was developed to predict next day's stock based on history without the knowledge of current market. Error percentage was around 1.5%. Main factors in the model were number of layers, neurons and connections, activation function in each neuron and training

algorithm. The input parameters were Lowest, Highest, Average value in d days. Mean Absolute Deviation, Mean Absolute Percentage Error, Mean Squared Error, Root Mean Squared Error were used as the criteria for evaluation. Elman was found to be better than MLP but it suffered greater errors.

Li and Liu in [11] determined network structure and prediction precision. The sample data was divided into training and testing data. The network was trained in order to fix study sample time order as much as possible. Testing sample data was used to test the trained network. The network giving satisfactory result was used for prediction. Weekly closing price from Shanghai Stock Exchange Centre was used as the dataset. The data was treated by sampling standard, sampling normalisation, number of nodes in hidden layer and LM algorithm to modify weight value and threshold.

Schierholt and Dagli in [12] focused mainly on maximising the performance of portfolios rather than maximising the percentage of correct decisions. The dataset used was 400 patterns of Standard and Poors (SNP) 500 index. Only the Closing values were considered. One of three outputs namely buy, sell or keep the current status was given. The two models used were MLP with back propagation and probabilistic NN. MLP worked in static environment with separate training and testing phases whereas probabilistic NN performed better in all the cases.

Hassan and Nath in [13] used Hidden Markov Model to locate patterns from past data sets that matched the current day's stock price behaviour. These two data sets were then interpolated with appropriate neighbouring price elements. $\lambda = (A, B, \pi)$ was the overall HMM model, where A is the transition matrix consisting of transition probabilities from state i to state j. B is the observation emission matrix and π is the prior probability matrix. Forward backward algorithm was used to compute the probability of occurrence of observation sequence. Viterbi algorithm was used to choose state sequence best explaining the observations. Baum Welch algorithm was used to train HMM. Input features were Open, Close, High, Low. The next day's closing value of the respective stock was given as the output.

Xiang and Fu in [14] predicted the stock market using multiple models. In this study the S&P 500 return rates are predicted instead of the index as the return rates are more stationery over time. The novel and significant part of this work is in the use of each NN to model a particular trend which are Bear, Choppy and Bull market trends. A second significance of the work is that an attempt is made at evaluating the trading performance considering commission costs which are important when predictions on higher frequency data are made. The model outputs next day's closing value. The metrics used for performance evaluation are Directional Accuracy and Annual Return rate. The 3 NN system is more consistent in getting better performance than other systems.

III. TECHNIQUES

A. Markov Chains

Everything is a Markov chain! For the purposes of our discussion, we will be considering first-order Markov chains that satisfy this property. Assume that $S = \{S_1, S_2, \dots, S_n\}$ is the set of all possible states and $X = \{X_k \mid X_k \in S, k = 1, \dots, T\}$ be the time series of states. Now, we also know that the Markov property is: For any $k \geq 0$ and any states $X_0 \dots, X_k$:

$$\begin{aligned} P(X_{k+1} = S_j \mid X_k = S_i, X_{k-1}, \dots, X_0) \\ = P(X_{k+1} = S_j \mid X_k = S_i) \end{aligned} \quad (1)$$

That is, the probability of smoothly transitioning to a certain state S_j at time step $k + 1$ depends solely on the current state S_i at time step k and not on any earlier states. This is why the probability distribution of the Markov chain at any future time step can be completely determined by its current state. You can formally define a first order Markov chain by giving the set of states S and a transition probability matrix $A = [a_{ij}]$, (with a_{ij} represents the possibility to transit from state S_i to state S_j in one step).

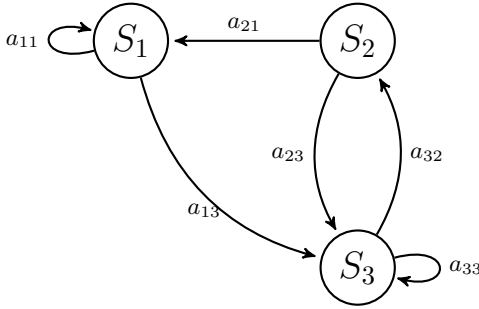


Fig. 1: Example of a Markov Chain with three states.

The transition matrix at each time should follow the standard stochastic conditions:

- $0 \leq a_{ij} \leq 1 \forall i, j$.
- $\sum_{j=1}^n a_{ij} = 1 \forall i$.

Let $x_k \in R^n$ be the vector of state probabilities at time k , then the system evolves according to:

$$x_{k+1}^T = x_k^T A \quad (2)$$

The starting set of state probabilities is referred to as $\pi = \{\pi_1, \pi_2 \dots \pi_n\}$.

B. Hidden Markov Models

The above formulation implicitly assumes that every state is observable (although that could be made more concrete) and hence each event can be mapped to some state. Markov chains are ubiquitous in modeling sequence data across different fields. Unsurprisingly, however, the world is full of hidden states that drive the observations but are not observed directly. This shortcoming gave birth to Hidden Markov

Models (HMMs), which upgrade our elementary Markov chain model by incorporating hidden states we can not see that in turn affect the observed data.

Layer-wise the hidden state process of a HMM is a Markov chain, and within each state observations are generated where each classes of observations has its own probability distribution that more often than not only depends on the class.

At a high level, we can say $O = \{O_k \mid O_k \in O, k = 1, \dots, T\}$, where O is the set of all possible observations. These hidden states evolve according to Equation 2.

The probability of symbol $o \in O$ being emitted by state S_i is captured by the emission probability function $b_i : o \mapsto b_i(o) \in [0, 1], \forall i$.

We will use emission probability matrix $B = \{b_{ij} = b_i(o_j)\}$ both in the current and subsequent sections.

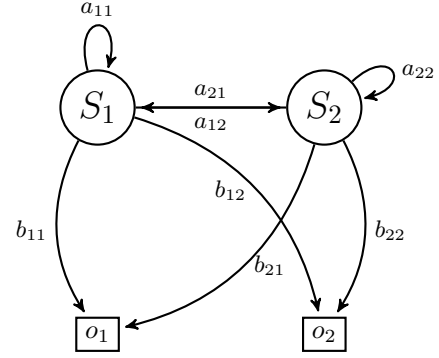


Fig. 2: Hidden Markov Chain Example with two states and two possible outputs.

C. Applications of HMMs: Common Problems and Solutions

Three main problems that are typically solved with HMMs are evaluation, decoding, and parameter estimation. The literature has broad related research on these problems and their solutions [15]. We provide a summary of these issues and their fixes in the subsequent paragraphs.

- **Evaluation Problem:** The evaluation problem in HMMs is the probability of having a specific observation sequence based on the model. That is, for a hidden Markov model with state transition parameters π, A , and output observation probabilities B , on input an observation sequence O , we want to compute $P(O \mid \pi, A, B)$.

The way of solving this is to calculate its forward variable $\alpha_k(i)$, which is the probability of being in state S_i at time k and observing till time k (sequence up to time k observe), calculated from the recursively on each time step k and state S_i .

At the initialization stage, the forward variable for the first time step, $k = 1$, is calculated as: $\alpha_1(i) = \pi_i b_i(O_1), \forall i$ where π_i is the initial probability of being in state S_i and $b_i(O_1)$ is the emission probability for observation O_1 .

At every subsequent time step $k > 1$, $\alpha_k(i)$ is calculated as the total probability of all paths that could have generated the observation up to time k and transition onto state S_i at this time. This can be expressed with the following equation:

$$\alpha_k(i) = \left(\sum_{j=1}^n \alpha_{k-1}(j) a_{ji} \right) b_i(O_k), \forall i \quad (3)$$

The algorithm solves this problem by calculating the forward variables once again, however, now for each possible time step, hence evaluating the probability of observing whole sequence based on the HMM parameters.

Finally, in our case to get the total probability of observing a sequence we sum up over all possible states i at time $t = T$.

$$P(O \mid \pi, A, B) = \sum_{i=1}^n \alpha_T(i) \quad (4)$$

- The Decoding Problem: The decoding problem is determining the most likely sequence of hidden states given an observation sequence and a Hidden Markov Model. In other words, we are interested in finding the hidden state sequence X which maximizes $P(X \mid O, \pi, A, B)$, where Given an HMM with the parameters π, A , and B , and an observed sequence O .

The decoding problem is usually solved with the Viterbi algorithm.

At each time step, the algorithm iteratively computes the paths to each state by leveraging both current observation and previous states probabilities.

It calculates the probability of being in every state at a particular time step and, additionally, keeps track of the most likely path to that state.

- Parameter Estimation Problem: Also Fitting the model parameters such that the probability of the observed sequence is maximum. For a given observed sequence O , we like to estimate the best values for parameters π, A , and B , that would maximize $P(X \mid O, \pi, A, B)$.

The solution to this problem is usually solved using the forward-backward algorithm (else Baum-Welch).

Baum-Welch algorithm maximum likelihood solution based on Expectation–Maximization (or EM) tailored for a HMM. It does so by carrying out 3 main steps: Forward, Backward and Update Step. The forward step, where the algorithm calculates the likelihood of each state at every time given everything observed until that point. Previously we described this as the evaluation problem. The backward step computes the probability of observing the remaining part of the sequence from each possible state at each time step. The backward procedure complements for the forward procedure in calculating what is known as the backward variable, denoted $\beta_k(i)$, that represents the probability of being in state S_i at time k and observing the rest of the sequence (since time $k + 1$

and up to some maximal time). This backward variable is recursively calculated for each time step k and state S_i . In the initialization step, the backward variable for the last time step is $\beta_T(i) = 1, \forall i$.

At every time step $k < T$, we calculate the backward variable $\beta_k(i)$ in the following manner that models a sum of probabilities of all paths possible from state S_i at time k to the end. That looks like this:

$$\beta_k(i) = \sum_{j=1}^n \beta_{k+1}(j) \cdot a_{ij} \cdot b_j(O_{k+1}) \quad (5)$$

The forward and backward variables are then used to adjust the parameters of the HMM given the observed sequence. Specifically, let $\xi_k(i, j)$ be the probability of being in state S_i at time k and state S_j at time $k + 1$, given the model and the observation sequence. It can be computed as:

$$\begin{aligned} \xi_k(i, j) &= \frac{P(X_k = i, X_{k+1} = j, O \mid \pi, A, B)}{P(O \mid \pi, A, B)} \\ &= \frac{\alpha_k(i) \cdot a_{ij} \cdot b_j(O_{k+1}) \cdot \beta_{k+1}(j)}{\sum_{r=1}^n \sum_{h=1}^n \alpha_k(r) \cdot a_{rh} \cdot b_h(O_{k+1}) \cdot \beta_{k+1}(h)} \end{aligned} \quad (6)$$

Define now $\gamma_k(i)$ as the probability of being in state S_i at time k , condition to the observation sequence and the model :

$$\begin{aligned} \gamma_k(i) &= P(X_k = i \mid O, \pi, A, B) \\ &= \frac{\alpha_k(i) \cdot \beta_k(i)}{\sum_{j=1}^n \alpha_k(j) \cdot \beta_k(j)} \end{aligned} \quad (7)$$

We can relate those quantities:

$$\gamma_k(i) = \sum_{j=1}^n \xi_k(i, j) \quad (8)$$

In which $\sum_{k=1}^{T-1} \gamma_k(i)$ stands for the transition count from the state S_i and $\sum_{k=1}^{T-1} \xi_k(i, j)$ stands for the transition count from S_i to S_j .

The second is the update step, which gives new values to initial state probabilities, transition probabilities and emission probabilities. Initial State Probabilities: Updated values are normalized forward-backward probabilities at first time step.

$$\pi_i = \gamma_1(i) \quad (9)$$

The transition probabilities are reweighted by the average number of times you expect to see each event and (interestingly) each state in the indices for those elements.

$$a_{ij} = \frac{\sum_{k=1}^{T-1} \xi_k(i, j)}{\sum_{k=1}^{T-1} \gamma_k(i)} \quad (10)$$

To estimate the emission probability $b_j(o)$ –the probability of observing symbol o from state S_j we evaluate how many times are expected to o be emitted from state

j normalizing by how many transitions are expected to j :

$$b_j(o) = \frac{\sum_{k=1}^T (\gamma_k \cdot \delta(O_k, o))}{\sum_{k=1}^T \gamma_k(j)} \quad (11)$$

where $\delta(O_k, o)$ is a Kronecker delta function given by 1 if $O_k = o$, and 0 otherwise.

The parameters of the model are trained through this process until they can not be revised further (convergence), what leads to a maximum likelihood of the sequence.

Carefully selecting the initial conditions is paramount, as it can have an important influence on the estimation of the HMM parameters.

The Baum-Welch algorithm, however, is more likely to converge on some set of parameters that accurately model the underlying dynamics so long as proper initial conditions are chosen.

D. Gaussian Mixture Models

Gaussian Mixture Models (GMMs) are useful for building complex probability distributions. The components of a GMM are weighted sums of multiple Gaussian distributions, each of which represents a subpopulation within the data.

Multivariate mixture models use GMMs as multivariate data. In a GMM, each Gaussian component corresponds to a multivariate distribution with its own mean vector and covariance matrix. The probability density function of a multivariate GMM is:

$$p(y) = \sum_{i=1}^K c_i N(y | \mu_i, \Sigma_i) \quad (12)$$

C_i is the weight of the i -th component, and $N(y | \mu_i, \Sigma_i)$ is a normal distribution with mean vector μ_i and covariance matrix Σ_i .

The weights C_i satisfies the constraint $\sum_{i=1}^K C_i = 1$ denoting that the probabilities sum to one. These weights define the strength of each Gaussian component in the distribution

Since GMMs can effectively capture the structure and heterogeneity in multivariate data containing multiple modes or clusters, modeling such complex structures are a key area of application for them. Hidden Markov Models use GMMs to initialize emission probabilities of hidden states. Building the GMM is done by fitting to our training dataset: the training algorithm will estimate the parameters of each Gaussian component, namely mean vectors and covariance matrices. These parameters are used to initialize the emission probabilities for each of those hidden states.

IV. TECHNICAL IMPLEMENTATION

In our experiment, we used the following three functions from the Statistics and Machine Learning ToolboxTM in MATLAB to train and test the HMM: `hmmtrain`, `hmmdecode` and `fitgmdist`.

We train and test the model on freely available on the web historic daily prices of Apple, IBM, and Dell stocks. Each observation in our dataset consists of three different

TABLE I: Number of Discrete Points Each Dimension

Variable	Number of Point
fracChange	$n_{fC} = 50$
fracHigh	$n_{fH} = 50$
fracLow	$n_{fL} = 50$

values representing the fractional change, fractional high, and fractional low for the day.

$$O_k = \left(\frac{Close - Open}{Open}, \frac{High - Open}{Open}, \frac{Open - Low}{Open} \right) \\ O_k = (fracChange, fracHigh, fracLow) \quad (13)$$

The three dimensional array O_k is a matrix of actual values. Since the probability of guessing any actual value is mathematically zero, some discretization of the observations was necessary. The number of points used for the discretization are summarized in Table I.

We use dynamic edges for discretization. For every train we are calculating the max and min of *fracChange*, *fracHigh* and *fracLow*. Then for the edges we will create three linearly spaced vectors using min as the lowest values and max values. Lastly, we assign its index in the edges array to every discrete value.

Moreover, compatibility with our monovariate Hidden In the case of Markov Model, it mapped the three-dimensional space to one-dimensional space. Enumerations of mappings were carried out according to Equation 14 that had described the points in the discrete set of three-dimensional space. As an inverse mapping, Equations 15 have been used:

$$n = (z - 1) \cdot (x_{max} \cdot y_{max}) + (y - 1) \cdot x_{max} + x \quad (14)$$

$$z = \left\lfloor \frac{n - 1}{x_{max} \cdot y_{max}} \right\rfloor + 1 \\ y = \left\lfloor \frac{n - 1 - (z - 1) \cdot x_{max} \cdot y_{max}}{x_{max}} \right\rfloor + 1 \\ x = (n - 1) \bmod (x_{max}) + 1 \quad (15)$$

A. Training

In the first trains, we used four latent hidden states underneath from which each of them outputs in the form of a GMM having four components. We re-trained our model with varying values. The relevant parameters for each train can be checked on our GitHub repository [16]. The parameters of these GMMs were estimated through the MATLAB function *fitgmdist*, optimizing the model along the EM algorithm. Parameters of GMM are first learned by the means of $k - means$ clustering.

We assumed the outcome probability density function as our initial state of the emission matrix. On the other hand, for transition matrix, we establish an initial estimation based on uniform distribution of probabilities.

The training data of HMMs are based on capitalizing on a rolling window method. Here, every observation sequence is the same, fixed at 10 days long. We will call this length *latency*. As it runs, the window moves incrementally over

the training period: the first sequence takes snapshots of observations at the beginning point, and for every subsequent sequence It displays new observations by advancing the window one day. The vector now reads all of the data set into the MATLAB function, *hmmtrain*. This function applies the Baum-Welch algorithm in order to estimate the transition and emissions matrices. As discussed above, the initial guesses used to initialise these estimations were.

B. Prediction

After the training, we conducted testing of our model for prediction of closing stock prices over various time horizons during the prediction period. The procedure for prediction for day d was the following:

- We accounted for the last $latency - 1 = 9$ observations available. Such observations preceded the following 9 days.
- Then we added each possible output for the current day d , constructing a sequence of 10 days. The sequence now has the 9 historical observations as well as one possible observation for the next day. There are $n_{fc} \cdot n_{fH} \cdot n_{fL}$ possibilities for the current day.
- We calculated the probabilities for each of the sequences to be developed by our supervised model. Finally, we selected select the observation with the greatest emission probability as an observation for the day.

In some cases, this probability of emitting historical observations together with a hypothetical observation becomes 0 or close to 0. This may either be due to true circumstances or some constraint on the model. However, we see improvements in our model in the *dynamicwindow*.

We have modified the prediction to address the issue algorithm: Based on the best probability achieved, if it is 0, we invoke the prediction algorithm again with reduced latency step by step, one day at a time, since decreasing latency enhances the chances of getting a feasible solution with emitted probabilities positive and keep reducing latency in such a way that we have some solution; also the historical sequence should be as long as possible. Thus, for our example we have determined that the historical series must include at least four days.

V. RESULTS

We had several metrics to evaluate the performance of our models. Mean Absolute Percentage Error, or MAPE, is Using the closing stock prices between the actual and predicted values.

$$MAPE = \frac{1}{n_p} \sum_{i=1}^{n_p} \frac{|P_i - c_i|}{|c_i|} \cdot 100\% \quad (16)$$

The forecasted stock close price, p_i is acquired on day i , where n_p is the total number of predictions and c_i the actual closing price of the stock. Table II Comparing our results on two different stocks versus the results in other papers [17], [18].

This task is hard enough to forecast the direction in which the price of the stock will move for a day. The next day, obviously is much more complicated and sometimes impossible

TABLE II: Comparison Of Mean Absolute Percentage Error

Stock Name	Our HMM	HMM-MAP [18]	HMM + Fuzzy Model [17]	ARIMA ANN
Apple Inc.	1.50	1.51	1.77	1.80
IBM Corp.	0.68	0.61	0.78	0.97

TABLE III: Directional Prediction Accuracy

Stock Name	MAPE	DPA	Training	Testing
Apple Inc.	1.50%	52.11%	2003-02-10 2004-09-10	2004-09-13 2005-01-21
	1.73%	63.27%	2003-02-10 2004-09-10	2004-10-13 2005-01-21
	1.05%	53.06%	2021-01-04 2022-01-03	2023-01-03 2023-06-30
	1.01%	40.57%	2017-01-03 2019-01-03	2023-01-03 2023-06-30

to forecast consecutively. Knowing that we are most likely able to predict if the value of the stock is going up or coming down throughout the day, we developed a new metric for its measurement called Directional Prediction Accuracy or DPA. DPA gives the percentage of correct directional predictions thus giving some precious information about how accurate our model is.

$$DPA = \frac{1}{n_p} \sum_{i=1}^{n_p} \delta(sgn(p_i - s_i), sgn(c_i - s_i)) \cdot 100\% \quad (17)$$

In Equation 17, n_p is the number of predictions, δ is the Kronecker delta function, p_i is the predicted stock closing price, c_i the actual closing price, and s_i the stock opening price. Table III emphasizes how MAPE and DPA do not measure equivalent quantities since, generally, they are not correlated.

Testing period 2004-10-13 to 2005-01-21 The dots represent the actual closing prices of the day. As long as they are green, the dots contribute towards DPA. Only then do the dots contribute towards the MAPE when they turn close to the actual closing value of the day.

VI. CONCLUSIONS AND FURTHER DEVELOPMENTS

We implemented the full version of the originally proposed stock prediction hidden Markov model by Gupta et al. [18]. We searched for some insight into its performance over

Fig. 3: Actual Stock Price for Training Data

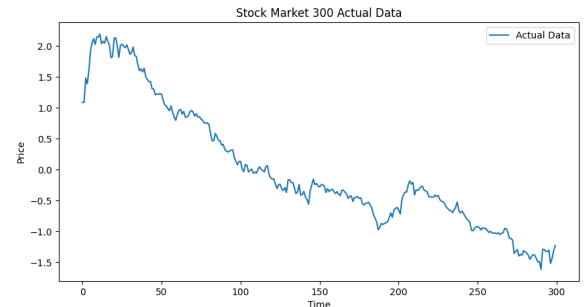


Fig. 4: Stock Price for Testing Data

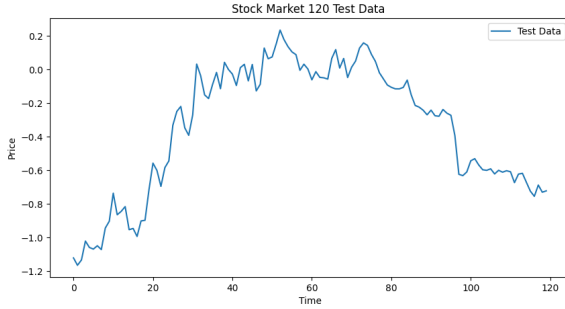


TABLE IV: Predicted Price

Close	Open	High	Low
318.40	288.31	285.99	309.30
301.18	283.68	307.41	289.79
309.61	326.11	311.03	300.55
264.89	252.28	248.97	239.36
249.60	235.094	261.78	256.76
251.65	241.58	247.82	286.90
229.90	255.17	257.16	236.43
249.81	253.55	286.17	273.11
246.66	271.95	214.93	263.56
264.24	262.08	263.90	242.35
254.62	269.63	283.37	266.43
232.56	267.02	291.67	245.41
252.29	259.58	273.75	273.90
228.03	250.36	240.65	270.44
270.99	264.20	247.24	235.72
259.93	268.75	262.77	234.08
258.83	227.99	250.33	243.66
261.37	261.43	263.01	268.95
258.92	249.32	195.89	260.90
257.46	267.89	265.90	224.12

Fig. 5: Comparison of Actual Open Price and Tested Open Price

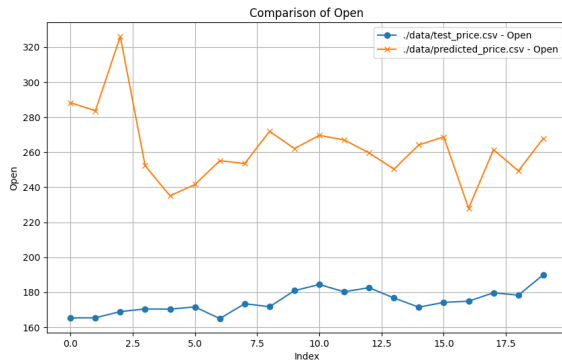


Fig. 6: Comparison of Actual Close Price and Tested Close Price

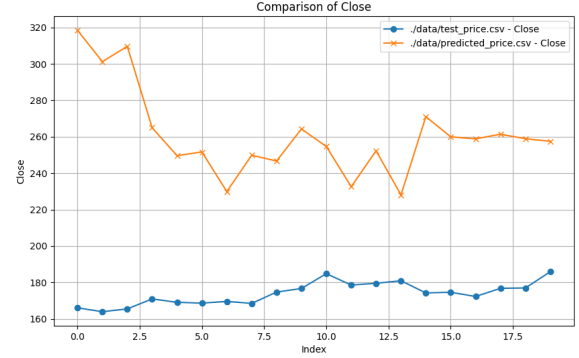


TABLE V: Test Price

Close	Open	High	Low
166.07	165.38	167.53	164.39
163.94	165.45	168.35	162.73
165.48	169	169.08	165.28
170.95	170.49	172.57	169.55
169.1	170.4	171.92	169.02
168.63	171.66	171.79	167
169.6	165	170.09	163.33
168.49	173.49	174.49	168.3
174.72	171.76	174.78	170.42
176.69	180.95	182.39	176.27
184.82	184.46	184.94	181.21
178.58	180.29	180.33	177.03
179.55	182.66	182.8	177.7
180.94	176.73	181.29	174.93
174.24	171.51	174.77	170.88
174.62	174.24	174.95	170.26
172.29	174.94	176.6	172.1
176.78	179.73	180.27	176.55
176.98	178.37	181.47	175.51
185.86	190	190.7	184.99

variant datasets while verifying the proper applicability in the scenario of stock market movement forecasting. After careful replication and extension of their approach, we have done an exhaustive evaluation with benchmark models to actually understand what kind of strength and weakness the model possesses.

Our model was trained on period of one to two. It can be verified using adaptability and reusability for the validation in terms of years and used for prediction at another timescale. Full source code along with characteristics of the pre-trained models are available at GitHub [16].

The proposed method was tested on several stocks, and the results were compared with those in HMM-MAP, HMM-Fuzzy, ARIMA, and ANN models. A complete hyperparameters exploration was conducted regarding its influence in achieving the optimal value. The

The outputs produced a much lower Mean Absolute Percentage Error compared to the HMM-Fuzzy, ARIMA, and ANN models trained on the same years as those reported in [18] and [17]. Moreover, to get an even broader perspective of how well our model was performing, we created a new measure known as the Directional Prediction Accuracy, DPA. The DPA

allowed us not only know our accuracy in simulating real movements for the stock price but also provided an extremely useful method for measuring performance. We therefore advocate for the tuning of the Emission and Transmission matrices over the latest time window every day prior to the actual forecast. From that, we first proposed updating the matrices by training the base model on the historical data. Using data from a fairly more recent period before making

All predictions would depend on being calibrated by data from 2022-06-01 up to 2023-06-01 for the matrices in advance of making any prediction for 2023-06-02, for instance, to train the main model from 2021-01-01 up to 2023-01-01. This would capture recent market trends and better predictions. Conclusion: Our implementation of Stock Prediction HMM showed good prediction capability. Fine-tuning gives us several potential improvements in the future for predictions and is left to be "warranted". Further research to capitalize on recent market trends. Our work adds to the body of research on predicting stock markets, offering significant contributions to traders and financial analysts.

REFERENCES

- [1] "Institutional investors definition— nasdaq." [Online]. Available: <https://www.nasdaq.com/glossary/i/institutional-investors>
- [2] "Investor bulletin: Hedge funds." [Online]. Available: <https://www.investor.gov/introduction-investing/investing-basics/investment-products/private-investment-funds/hedge-funds>
- [3] "Algo or algorithmic trading definition — nasdaq." [Online]. Available: <https://www.nasdaq.com/glossary/a/algo-trading>
- [4] Aditya Gupta and Bhuwan Dhingra, "Stock Market Prediction Using Hidden Markov Models", IEEE 2012.
- [5] Kumar Abhishek, Anshul Khairwa, Tej Pratap and Surya Prakash, "A stock market prediction model using artificial neural network", ICC-CNT'12, 26th– 28th July 2012, Coimbatore, India.
- [6] F.Luo, J. Wu and K. Yan, "A Novel Nonlinear Combination Model Based on Support Vector Machine for Stock Market Prediction", Proc. Of the 8th World Congress on Intelligent Control and Automation, July 6-9 2010, Jinan, China.
- [7] J.H.Wang and J.Y.Leu, "stock market trend prediction using ARIMA-based neural network," Proc. Of IEEE conference on neural networks, volA, pp.2 160 -2 165, 1996
- [8] Akinwale Adio T, Arogundade O.T and Adekoya Adebayo F, "Translated Nigeria stock market price using artificial neural network for effective prediction. Journal of theoretical and Applied Information technology , 2009.
- [9] F. Wang, L. Liu and C. Dou, "Stock Market Volatility Prediction: A Service-Oriented Multi-Kernel Learning Approach", IEEE 9th International Conference on Services Computing, 2012.
- [10] M. Naeini, H. Tarehian and H. Hashemi, "Stock Market Value Prediction Using Neural Networks", International Conference on Computer Information Systems and Industrial Management Applications (CISIM) 2010
- [11] F. Li and C. Liu, "Application Study of BP Neural Network on Stock Market Prediction", 9th International Conference on Hybrid Intelligent Systems 2009.
- [12] K. Schierholt and C. Dagli, "Stock Market Prediction Using Different Neural Network Classification Architecture", Computational Intelligence for Financial Engineering 1996
- [13] Md. Rafiul Hassan and Baikunth Nath, "Stock Market forecasting using Hidden Markov Model: A New Approach", Proceeding of the 2005 5th international conference on intelligent Systems Design and Application 0-7695-2286-06/05, IEEE 2005.
- [14] C. Xiang and W.M. Fu, "Predicting the Stock Market using Multiple Models", ICARCV 2006
- [15] Rabiner, L. R. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. Proceedings of the IEEE, 77(2), 257-286. <https://doi.org/10.1109/5.18626>
- [16] (2023) valentinomario/HMM-stock-market-prediction. [Online]. Available: <https://github.com/valentinomario/HMM-Stock-Market-Prediction>
- [17] M. R. Hassan, "A combination of hidden markov model and fuzzy model for stock market forecasting," Neurocomputing, vol. 72, no. 16, pp. 3439–3446, 2009.
- [18] A. Gupta and B. Dhingra, "Stock market prediction using hidden markov models," in 2012 Students Conference on Engineering and Systems, 2012, pp. 1–4.
- [19] Baum, L. E.; Petrie, T.; Soules, G., Jr; Weiss, N. (1970). A maximization trick that happens when we get to the statistics of probabilistic functions on Markov chains. Annals of Mathematical Statistics, 41(1), 164–171. Association for Computing Machinery: <https://doi.org/10.1145/1665737> HagertyS., Kumar V., ShahC. · AIT uses C.; Several ISI Masters in Big Data →AOMS/[http s:/ /d oi.or g/1 0.12 14 /aom s/1 7 76 97](http://s:/d oi.or g/1 0.12 14 /aom s/1 7 76 97) (2019)
- [20] Luigi Catello, Ludovica Ruggiero, Lucia Schiavone, and Mario Valentino, *Hidden Markov Models for Stock Market Prediction*, 5 October 2023.
- [21] Bram de Wit, Supervisor: Prof. Dr. L.R.B. Schomaker, *Stock prediction using a Hidden Markov Model versus a Long Short-Term Memory*.
- [22] Raihan Tanvir, Md Tanvir Rouf Shawon, Md. Golam Rabiul Alam: Ahsanullah University of Science and Technology, Dhaka, Bangladesh: Brac University, Dhaka, Bangladesh, *DSE Stock Price Prediction using Hidden Markov Model*, 26 January 2023.
- [23] Ravindra Nath, Renu Jain, CONF IEEE Using randomized search algorithms to estimate HMM learning parameters, 2009 IEEE International Advance Computing Conference DOI: 10.1109/IADCC.2009.4808999
- [24] Ravindra Nath Atmaprakash Singh Santosh Kumar, Peer reviewed open access Handwritten English Character Recognition using HMM, Baum-Welch and Genetic Algorithm, 02-04 November, 03 January 2019 ISBN Information: INSPEC Accession Number: 18384376 DOI: 10.1109/UP-CON.2018.8596954
- [25] Jain Renu, Ravindra Nath, Proceedings of the Second International Conference on Soft Computing for Problem Solving (SocProS 2012), December 28-30, 2012, Springer, New Delhi ISSN 2194-5357 https://doi.org/10.1007/978-81-322-1602-5_1
- [26] Parameter Estimation of Hidden Markov Models (HMM) using Go With the Winner Algorithms, Ravindra Nath, Renu Jain - International Journal of Computer Applications, 2011